

Análisis curricular de trayectorias estudiantiles con enfoque dirigido por modelos (MDE) y minería de procesos

Modulo Taller Docente Andrea Delgado - Facultad de Ingeniería, Udelar - 2025

Santiago Nicolás Díaz Conde
Facultad de Ingeniería
Universidad de la República
santiago.nicolas.diaz.conde@fing.edu.uy
0009-0005-5626-5096

Abstract—Este trabajo propone una extensión orientada a la Minería de Procesos (PM) aplicada al modelado y análisis de currículas y trayectorias académicas.

La disponibilidad de herramientas destinadas a realizar minería y análisis de currículas y trayectorias es limitada[2]. Este proyecto surge en respuesta a las carencias identificadas en las soluciones existentes, con el propósito de brindar una solución dinámica que permita una mayor cantidad de análisis curriculares.

Aunque existen múltiples estudios con enfoque en minería de procesos aplicada a currículas, el software especializado es prácticamente inexistente, y además, el software existente que permite realizar dichos análisis requieren de un conocimiento avanzado en el área de minería de procesos, algo que genera una barrera para poder realizar análisis de currículas y trayectorias que utilicen técnicas de minería de procesos sin tener conocimientos en el área.

En este contexto, este proyecto busca abordar estas limitaciones y proporcionar una solución más eficiente y efectiva. El objetivo es desarrollar una herramienta interactiva y dinámica que permita a los estudiantes, docentes y personal administrativo de diversas disciplinas académicas visualizar y analizar currículas y trayectorias de carreras. Este proyecto no solo mejorará la experiencia académica de los estudiantes al diseñar su trayectoria, o la experiencia de los docentes y los funcionarios al analizar las unidades curriculares, sino que también les permitirá tomar decisiones informadas y planificar a futuro diferentes acciones a tomar de manera más eficiente. Al hacerlo, este proyecto tiene la capacidad de potenciar la forma en que los estudiantes interactúan con su trayectoria académica, mejorando su capacidad para alcanzar sus metas y tener éxito en su recorrido académico. Como además de que docentes y funcionarios puedan mejorar y enfocarse en las problemáticas de los planes de estudio de las diferentes carreras más fácilmente.

Entre sus funcionalidades se incluye un *dashboard* que brinda métricas de unidades curriculares, análisis de desviaciones respecto a trayectorias sugeridas, comparaciones entre generaciones de estudiantes y otros indicadores relevantes. Para su implementación se empleó la librería de código abierto *PM4Py*, lo que garantiza flexibilidad y escalabilidad en el desarrollo.

Index Terms—Model Driven Engineering, Educational Process Mining

I. CONCEPTOS DE INGENIERÍA DIRIGIDA POR MODELOS Y MINERÍA DE PROCESOS

A. Ingeniería Dirigida por Modelos

La ingeniería dirigida por modelos (MDE) es un enfoque de desarrollo de software en el que se enfoca en modelos como parte central del proceso de desarrollo del sistema. En el cual se construyen modelos que representan el sistema, los cuales luego se pueden transformar en otros modelos o en código ejecutable. Esto permite elevar el nivel de abstracción, facilitando la comprensión, el análisis y la evolución de sistemas complejos. Dentro de este paradigma informático, podemos encontrar diferentes conceptos fundamentales:

- 1) Modelo: Una representación simplificada del sistema o la realidad, expresada mediante diferentes elementos que se relacionan entre ellos, que capturan las características del sistema o la realidad.
- 2) Meta-Modelo: Define como se construyen los modelos, definiendo su estructura, como son sus elementos válidos, reglas, relaciones, etc.
- 3) Transformación de Modelos: Es el proceso por el cual un modelo se convierte o transforma en otro modelo o código fuente, permitiendo cambiar el nivel de abstracción de lo representado.

En la aplicación se utiliza Eclipse Modeling Framework (EMF)[5], este framework permite construir modelos estructurados, y a partir de estos generar código de forma automática en java, como también manipularlos, y almacenarlos en su formato estandarizado xml/xmi[6] de forma sencilla, siendo Ecore[5] el meta-modelo con el que se realiza todo esto. Con elementos básicos como son clases, atributos, relaciones y tipos, que constituyen la base para la definición de meta-modelos de diferentes dominios. Permitiendo un incremento de la productividad y asegurando consistencia en la implementación.

La utilización de herramientas como EMF[5] aportan beneficios muy grandes en el desarrollo de software, esto es debido a que aportan varios factores importantes que son clave

para aumentar la calidad y eficiencia del proceso de desarrollo. En primer lugar brinda Automatización, la cual permite generar de forma automática código y artefactos a partir de modelos, reduciendo enormemente la cantidad de tareas y esfuerzo manuales y repetitivos. En segundo lugar garantiza la Consistencia, brindando control de coherencia entre modelos y código, evitando discrepancias, asegurando un desarrollo del sistema coherente. Así mismo, brindando Estandarización, con una adopción de estándares reconocidos para el almacenamiento, intercambio y evolución de las representaciones y modelos. Lo que facilita la interoperabilidad, el entendimiento del sistema en especial entre diferentes herramientas y realidades, como así para su desarrollo estandarizado y sencillo. Y por ultimo proveyendo un incremento en la Productividad y Reutilización al brindar la posibilidad de construir modelos o meta-modelos específicos de diferentes dominios y reutilizarlos en distintos escenarios, al igual que una mayor facilidad a la hora de desarrollar el sistema, brindando un desarrollo acelerado y sostenible a largo plazo.

En este contexto, MDE no solo contribuye a estructurar de forma clara el dominio de curriculas y trayectorias de los estudiantes, sino que habilita la creación de herramientas que pueden evolucionar de manera controlada y escalable a medida que se incorporan nuevas necesidades. Por esto, la utilización de este enfoque ha ayudado enormemente a un desarrollo ágil y eficiente de las herramientas y necesidades con las que se trabajó.

B. Minería de Procesos

La minería de procesos es un campo que combina ingeniería de datos y la ingeniería de procesos. Cuyo objetivo es descubrir, monitorizar, analizar y mejorar procesos a partir del registro de eventos (logs) que se generan por diferentes sistemas de información de diferentes organizaciones. Este enfoque brinda la posibilidad de tener una visión objetiva basada en los datos de como se ejecutan realmente dichos procesos, facilitando la comprensión y evaluación de estos procesos, permitiendo optimizarlos y mejorarlos.

Para esto, se identifican tres tipos de actividades fundamentales:

1) *Descubrimiento de Procesos*: El descubrimiento de procesos es una actividad que consiste en construir un modelo de forma automatizada utilizando diferentes algoritmos a partir de un registro de eventos, sin poseer conocimientos previos del mismo. El modelo resultante de esta actividad refleja como se ejecutan realmente los procesos dentro de una organización, identificando el flujo de las actividades, los posibles caminos alternativos que acontecen, sus bucles, excepciones, y desvío de los comportamientos de los procesos. Como también datos de como diferentes actores interactúan en los procesos y se comunican entre si. El descubrimiento de procesos aporta un gran valor al permitir visualizar procesos que muchas veces no están documentados o difieren de lo esperado, o conocer en profundidad como los procesos se ejecutan realmente. Los algoritmos mas utilizados

en esta actividad son principalmente el alpha miner, el heuristic miner y el inductive miner.

En cuanto a la solución desarrollada, esta actividad es importante para poder ver como los estudiantes realizan realmente su trayectoria académica, y poder realizar diferentes análisis al respecto, por ejemplo, visualizar el desempeño de diferentes unidades curriculares, las trayectorias mas utilizadas por los estudiantes, entre otros tipos de análisis.

2) *Análisis de Conformidad*: Esta actividad tiene como objetivo comparar los registros de eventos con respecto a un modelo existente (ya sean realizados manualmente o automáticamente a través del descubrimiento de procesos), al partir del cual se realizan diferentes análisis mas específicos, como es detectar en que medida los registros de eventos se adecúan al modelo existente, y en que lugares ocurren desviaciones en los mismos. Esta actividad es esencial para garantizar la calidad de los modelos construidos. También con este análisis es posible detectar violaciones a los modelos como son reglas y normativas de los mismos, incumplimiento de plazos, la realización de actividades no autorizadas, como también por ejemplo, desviaciones en el orden de ejecución de las actividades. Entre otros tipos de análisis. Permitiendo la identificación de problemas en los procesos, y una mayor transparencia y conocimientos de los mismos.

Esta actividad fue utilizada en la aplicación para poder visualizar desviaciones y cambio de orden de las unidades curriculares de la trayectoria de los estudiantes con respecto a la curricula sugerida realizada por el equipo académico de las instituciones académicas. Permitiendo un mejor conocimiento de como los estudiantes avanzan en su carrera.

3) *Mejora*: Este análisis consiste en buscar enriquecer o extender un modelo existente con información adicional obtenida de los registros de eventos. A diferencia del descubrimiento, no se parte de cero, sino que se parten de modelos ya definidos para optimizarlos y mejorarlos, de forma que sean mas útiles en la practica. Las mejoras se pueden dar en base a metricas de desempeño, por ejemplo, tiempos de espera, cuellos de botella, frecuencias comunes en la traza de los procesos, entre otras métricas. Gracias a esto, las organizaciones pueden tomar decisiones estratégicas o tácticas orientadas a la mejora de los procesos, como puede ser reducción de costos, incremento de eficiencia, etc. Apoyándose en evidencia concreta de los procesos durante su ejecución.

Con respecto a esta actividad, se espera que los administradores académicos utilicen los datos brindados por la herramienta para poder realizar dicha actividad por su propia cuenta, o si ellos lo desean utilizando otras herramientas (por ejemplo sugerencias utilizando inteligencia artificial), las cuales no están abordadas por esta herramienta desarrollada.

II. PROPUESTA INICIAL

El software original de lo que se partió es una plataforma web de creación, modificación y consulta de modelos de estudiante, facultades, carreras, planes y demás, con un enfoque de microservicios en base a Java Servlets[7]. Todo el procesamiento (en backend) se realiza con MDE mediante la

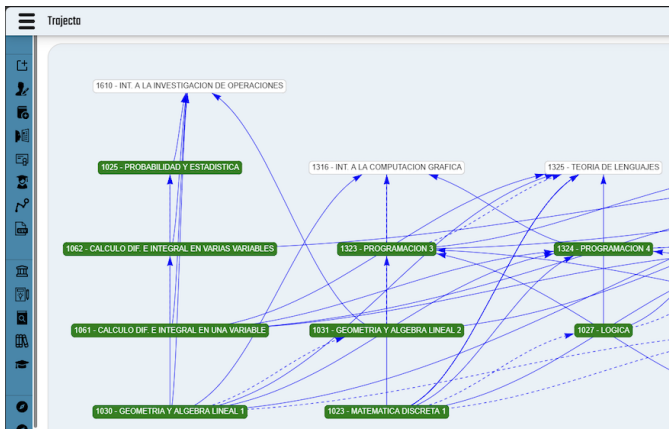


Fig. 1. Ejemplo de la funcionalidad ver trayectoria de un estudiante

librería EMF[5] en Java. Sin bases de datos, donde se hacen consultas o escrituras directas al modelo que se encuentra en el sistema de archivos del servidor y de allí se devuelven los resultados.

La propuesta inicial brindaba múltiples funcionalidades, cuyo fin era el permitir que un estudiante pudiese ver y analizar que unidades curriculares podía cursar en base a su trayectoria académica, así como poder modelar su trayectoria académica y conocer la curricula de un plan de estudios. Para esto se brinda la posibilidad de crear su propio modelo base, y a partir de este, empezar a agregarle diferentes actividades como son la inscripción a un plan, el cursado de unidades curriculares, como así también el resultado de exámenes realizados. También permitía ver datos varios de la curricula de las instituciones, por ejemplo que unidades curriculares son brindados por una institución, que unidades curriculares pertenecen a un grupo de materias, o por ejemplo los requerimientos de una unidad curricular, al igual que datos como por ejemplo la cantidad de créditos que brinda. O también visualizar la curricula entera de un plan en específico de una carrera. Brindando conocimientos varios para que el estudiante pueda tomar decisiones de que cursar durante su trayectoria. También implementa una Api utilizando OpenApi[4], para que terceros puedan realizar herramientas y estudios varios con la información brindada por esta.

III. IMPLEMENTACIÓN PROPUESTA INICIAL

A. Capa 1: Backend con Eclipse Modeling Framework (EMF)

En esta capa se definieron los modelos y metamodelos. En total existen dos metamodelos: el de instituciones y el de estudiante. Este último tiene referencias al primero, lo que permite separar la aplicación en dos actores, uno que es el estudiante quien crea su modelo, y otro que es el funcionario de la institución que es quien realiza los cambios en los modelos de la institución.

Para el funcionamiento de la aplicación se debe cargar, por parte del administrador del sistema, un modelo de instituciones con las respectivas carreras y las implementaciones de sus planes (materias, unidades curriculares, requerimientos de una

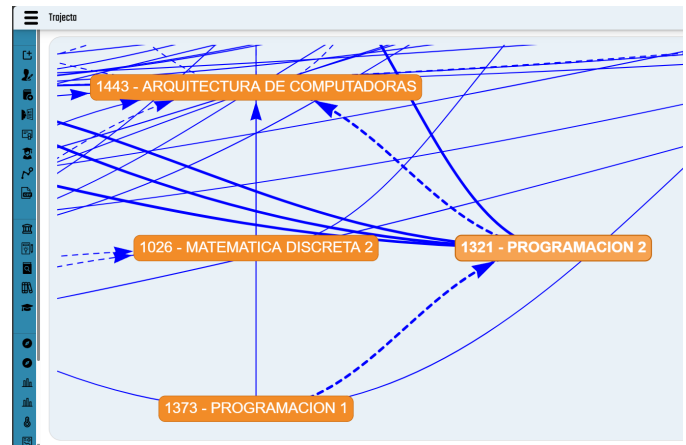


Fig. 2. Ejemplo de la funcionalidad ver curricula de plan

unidad curricular, evaluaciones, etc). A partir de este modelo -que debe cumplir con lo definido por el metamodelo- un estudiante puede generar su modelo con el que realizar las diferentes consultas que brinda la aplicación.

B. Capa 2: OpenAPI

A su vez, el software original poseía su comunicación entre Front-End y Back-End definidos por un modelo en OpenApi[4] de forma estandarizada, brindando una alta consistencia en la definición de las llamadas al servidor. Lo que permite una mas fácil modificación e implementación de cosas en la vista del usuario (Front-End), al igual que modificaciones en el Back-End con retro-compatibilidad.

Las operaciones se dividen en 2 grandes grupos, las GET y las POST, siendo que, las GET son principalmente para realizar consultas de información sobre el modelo de instituciones, mientras que las POST requieren de información extra para funcionar, en específico en general se necesita el modelo del estudiante, y con esto se pueden realizar consultas y modificaciones al modelo del estudiante. Como puede ser agregar evaluaciones, ver que unidades curriculares puede cursar un estudiante, entre otras funcionalidades.

En total son 12 operaciones: La aplicación cuenta con varias funciones clave para manejar modelos de estudiantes y consultar información académica. La función `addStudentEval` permite agregar una evaluación a un modelo de estudiante, ya sea de curso o examen, mientras que `addStudentPlan` se utiliza para agregar una inscripción a un plan dentro del modelo. La función `createStudentModel` facilita la creación de un nuevo modelo de estudiante o el agregar estudiantes nuevos a un modelo existente.

Para evaluar el progreso académico, `eval` permite analizar los requerimientos de un estudiante en una Unidad Curricular específica dentro del modelo, y `evalPlan` determina qué Unidades Curriculares puede cursar un estudiante según su situación actual. La función `evaluations` permite obtener todas las evaluaciones asociadas a un estudiante.

En cuanto a la información institucional, `getCarrera` obtiene datos de una carrera, como son sus materias, y unidades

Para ver lo que requiere y lo que retorna en específico de cada una de las operaciones, se recomienda ver la implementación en OpenApi[4] de la aplicación. Estas poseen una descripción mas exhaustivas de las tareas que realizan.

La capa de frontend representa la interfaz visible para el usuario y se encarga de la interacción directa con la aplicación. Esta capa ha sido desarrollada de manera estática utilizando HTML, CSS y JavaScript, garantizando una estructura ligera y de fácil mantenimiento. Para la visualización de trayectorias de estudiantes y de los diagramas curriculares de los planes de estudio, se integra la librería vis.js[8], la cual permite crear gráficos interactivos y representaciones dinámicas de los datos de manera intuitiva.

Aunque la capa front-end es relativamente sencilla desde el punto de vista arquitectónico, cumple un rol fundamental al facilitar el acceso a las funcionalidades de la aplicación. Permite a los usuarios explorar visualmente la información de los estudiantes, consultar el estado de sus planes académicos y analizar la estructura de los currículos de manera clara y comprensible. Además, la separación entre la lógica de presentación y la lógica de negocio asegura que la interfaz permanezca modular y adaptable, lo que facilitó las mejoras a realizar, actualizaciones o la incorporación de nuevas funcionalidades sin afectar la consistencia del sistema.

Se realizaron múltiples modificaciones en los meta-modelos de la solución original. En el meta-modelo de las instituciones se agregó el tipo de evaluación parcial, esto permite que un estudiante pueda registrar su progreso en diferentes unidades curriculares si lo desea, como también permitir a los administradores académicos y docentes obtener conocimiento sobre la evolución de los estudiantes, por ejemplo, sabiendo como el resultado de los parciales afectan la evolución y exoneración de los estudiantes o de una generación en específico, o también poder comparar diferentes ediciones de una unidad curricular y evaluar la evolución de su dificultad y calidad de enseñanza. Otra modificación que se realizó fue en el meta-modelo de los estudiantes, en este se agregó la posibilidad de que se pueda registrar la obtención de títulos de los estudiantes. Este agregado es interesante debido a que permite la posibilidad

```

classDiagram
    class Evaluation {
        +date : EDate
    }
    class Course {
        +Year : Elnt
        +Edition : Elnt
    }
    class CurricularUnit {
        +Id : EString
        +Name : EString
        +Cred : Elnt
        +Valid : EBoolean
    }
    class PartialEvaluation
    class ExamEvaluation
    class CourseEvaluation

    Evaluation <|-- PartialEvaluation
    Evaluation <|-- ExamEvaluation
    Evaluation <|-- CourseEvaluation

    Evaluation "0..*" -- "1..1" CurricularUnit : [0..*] partialevaluation
    Evaluation "0..*" -- "1..1" Course : [0..*] ExamEvaluation
    Evaluation "0..*" -- "1..1" Course : [1..1] course
    Evaluation "0..*" -- "1..1" Course : [0..*] CourseEvaluation
    CurricularUnit "1..1" -- "1..1" Course : [1..1] curricularunit
    CurricularUnit "1..1" -- "1..1" Course : [1..1] course
    CurricularUnit "1..1" -- "1..1" Course : [1..1] CurricularUnit
  
```

```

classDiagram
    class Student {
        Id : EString
        Name : EString
    }
    class PlanInscription {
        date : EDate
        Plan : Plan
    }
    class Degree {
        date : EDate
    }
    class CourseInscription {
        date : EDate
        CUCourse : Course
    }
    class StudentEvaluation {
        grade : EInt
        evaluation : Evaluation
    }

    Student "0..1" --> "0..1" PlanInscription : StudentPlanInscription
    PlanInscription "0..1" --> "0..1" Degree : degree
    PlanInscription "0..1" --> "0..1" CourseInscription : PlanCourseInscription
    PlanInscription "0..1" --> "0..1" StudentEvaluation : PlanStudentEvaluation

```

En esta capa muchos cambios fueron necesarios. Principalmente para poder dar soporte de mejor forma a las funcionalidades de minería de procesos, como también para poder extender mejor las funcionalidades existentes de la herramienta. Entre estas modificaciones se encuentran modificaciones a las operaciones `addStudentEval`, `addStudentPlan`, `eval`, `evalPlan` y `evaluations`, con el fin de poder brindar soporte de modelos de estudiante que posean multiples estudiantes. También se tuvieron que modificar las operaciones que incluían listar unidades curriculares (por ejemplo `getUCPlan`), para poder brindar la opción que ademas de brindar el id de la unidad curricular también se brinde el nombre de esta, esto con el fin de mejorar la usabilidad de la herramienta, por ejemplo que anteriormente era necesario conocer los id de las unidades curriculares para poder trabajar con ellas, por ejemplo para agregar una inscripción a una unidad curricular, o agregar una evaluación, mientras que con estas modificaciones es posible poder realizar búsqueda de las unidades curriculares, al igual que mostrarlas en base a su nombre unicamente. Mejorando enormemente la usabilidad de la herramienta.

Por otro lado, fue necesario agregar 3 nuevas operaciones

- 1) La operación `EstudianteAddCourse` que permite agregar una inscripción a un curso por parte de un estudiante. Esta funcionalidad ya se encontraba modelada en el metamodelo original, sin embargo, la definición e implementación de dicha funcionalidad no se encontraba desarrollada. Y es útil debido a que algunos requerimientos de unidades curriculares implica saber si un estudiante esta inscrito a una unidad curricular en específico. O también para poder tener la posibilidad de realizar minería de procesos de forma mas granulada, obteniendo la posibilidad de un análisis mejor.
- 2) Otra operación necesaria de desarrollar fue `GetStudentsLog` la cual se encarga de obtener un log en formato CSV a partir de un modelo de estudiantes, esto es necesario para que se puedan realizar diferentes funcionalidades de minería de procesos. Como por ejemplo el descubrimiento de procesos o el análisis de performance o conformidad con los datos de los estudiantes.
- 3) Finalmente, se agregó `EstudianteAddDegree` con el fin de brindar la nueva funcionalidad definida en la Capa 1 de poder agregar un titulo a un estudiante.

Todos estos cambios se realizaron de forma retro-compatible con la definición de OpenApi anterior. Por lo que, una aplicación, o un Front-end que utilizase una definición anterior, puede seguir utilizando la nueva definición modificada sin encontrarse con ningún problema ni tener la necesidad de realizar ninguna modificación. Lo cual es bueno ya que demuestra la capacidad de evitar discrepancias, manteniendo un desarrollo de sistema coherente que brinda la utilización del enfoque de Ingeniería Dirigida por Modelos, y la utilización de estándares como es OpenApi.

C. Capa 3: Frontend

Esta capa sufrió múltiples cambios enfocados en la usabilidad, uno de los cambios mas relevantes fue el soporte de múltiples idiomas, incluyendo de base los idiomas español e ingles, permitiendo que la herramienta pueda ser utilizado por un publico mas amplio. Por otro lado, otros de los cambios realizados fueron la posibilidad de realizar búsquedas en los selectores de unidades curriculares como a su vez, el mostrar el nombre de una unidad curricular y no unicamente su identificador a la hora de seleccionarlal. Esto se realizo agregando la librería `choices.js`[9] al front-end, permitiendo que un usuario no necesite conocer exactamente el id de una unidad curricular, sino que ademas, mejorar la búsqueda de estas a la hora de seleccionarlal, lo que en el diseño anterior, podía ser incluso frustrante en algunos casos. También se agregaron pantallas de carga en diferentes operaciones, con el fin de mostrar al usuario que utiliza diferentes funcionalidades que se esta realizando un trabajo y debe esperar a que finalice, y que la herramienta no se "tildó". Un ejemplo de esto es a la hora de mostrar la trayectoria de un estudiante. Todos estos cambios brindaron una mejoría en la usabilidad de la herramienta. Permitiendo que un usuario de esta pueda usarlas de forma mas natural y simple.

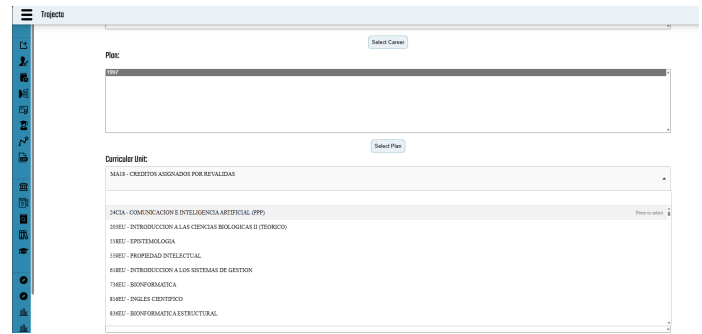


Fig. 5. Select mejorado con búsqueda de unidades curriculares

V. MINERÍA DE PROCESOS PARA ANÁLISIS CURRICULAR

Una vez realizados los cambios en la propuesta inicial, se procedió a trabajar en el agregado de la sección de Minería de Procesos a la herramienta.

Se decidió que se realizaría una arquitectura a base de micro servicios en Python que utilizarían la librería `Pm4Py`[3] para realizar las tareas de minería de procesos y `Flask`[10] para brindar las capacidades de la Api a través del protocolo HTTP. Siendo un total de 8 micro servicios que brindarían las capacidades necesarias para brindar el total de las 11 funcionalidades de la herramienta (siendo varias la misma funcionalidad pero una para el modelo de estudiantes y otra para el modelo de referencia, es decir la curricula sugerida).

A. Funcionalidades brindadas

La principal funcionalidad desarrollada fue la posibilidad de realizar el descubrimiento de modelos, utilizando diferentes algoritmos como `Alpha Miner`, `Heuristics Miner` o el `Inductive Miner`, esta funcionalidad se divide en dos funcionalidades de cara al usuario al igual que varias de las siguientes funcionalidades, ya que es posible realizar 2 descubrimientos diferentes, uno con los datos de los estudiantes, que representa la realidad de la trayectoria de los estudiantes, y otro para el modelo de referencia, que seria la curricula sugerida, con el que con otras funcionalidades se pueden utilizar dichos descubrimientos. También se brinda el poder ver diferentes estadísticas de los modelos descubiertos, tanto como puede ser de los modelos de referencia como para los modelos de los estudiantes, con datos como por ejemplo cantidad de variantes, actividades de inicio y final de las trayectorias, entre otros datos. Al igual que estadísticas de las unidades curriculares que si bien son mas orientados a la minería de datos, es interesante poder verlos, en especial para los modelos de estudiantes, con datos como ratios de exoneración por edición de curso, al igual que gráficos de mediana de notas, y gráficos de cantidad de recursadas de los cursos por parte de los estudiantes, entre otros datos varios que son interesantes. Y al igual, también datos de los diferentes planes de carreras, con datos por ejemplo como distribución de créditos de los estudiantes en dicha carrera. Otra funcionalidad fundamental brindada de minería de procesos, es el poder visualizar los descubrimientos realizados, permitiendo verlos por ejemplo en

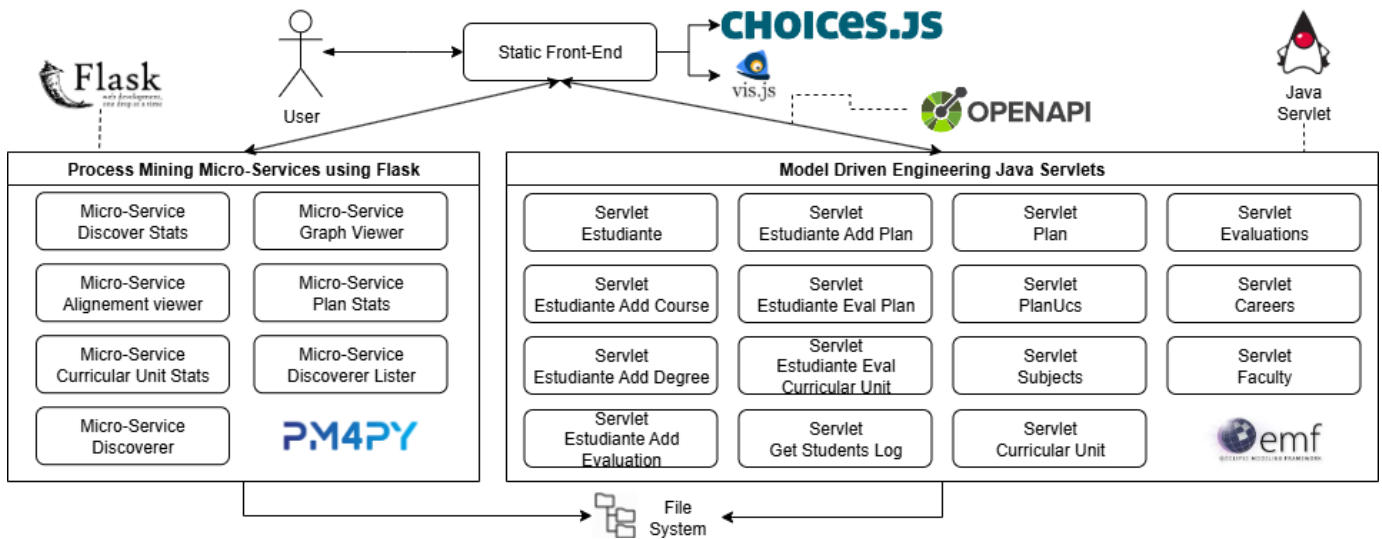


Fig. 6. Arquitectura final del proyecto

diagramas de petri-net, o diagramas de seguimiento directo. Además de poder ver la posibilidad de ver el rendimiento también en forma de diagrama de los diferentes modelos descubiertos. Y por último, otra funcionalidad es la de ver el diagrama de alineamiento, que es un tipo de estudio de conformidad, como se describió anteriormente, que permite ver donde los estudiantes se desvían con respecto a la curricula sugerida, donde hay bucles, entre otras desviaciones que pudiesen ocurrir, como son caminos alternativos que tomen los estudiantes. Los cuales permiten que los administradores académicos se centren en donde falla la curricula sugerida, o en que unidades curriculares se deben centrar en trabajar, entre otros tipos de análisis de mejora que se pudiesen realizar. También se llegó a desarrollar un estudio de análisis de conformidad utilizando los logs de estudiantes con respecto a la curricula sugerida, utilizando Replay Token on a Petri-Net. Sin embargo, debido a carencias en la librería Pm4Py[3], no era posible graficarlos ya que no existía un graficador en dicha librería, por lo que era necesario implementar uno de cero, por lo que, dicha funcionalidad si bien existe el micro servicio que lo realiza y brinda los datos, no fue posible implementarla en el Front-End para que pueda ser utilizada por parte de los usuarios. La cual brinda además de los datos para poder graficar la petri-net con datos como por ejemplo en que actividad se pierden mas tokens, también datos como son:

- 1) Fitness: Que es un indicador que mide que tan bien el modelo reproduce el comportamiento observado en el log, en un modelo en específico. Calculándose como:

$$1 - \frac{\text{Tokens perdidos} + \text{Tokens restantes}}{\text{Tokens consumidos} + \text{Tokens producidos}}$$

- 2) Precisión: Que es un indicador que mide que tan específico es un modelo respecto al log que representa la realidad, esto por ejemplo permite saber que tantos comportamientos permite un modelo con respecto a

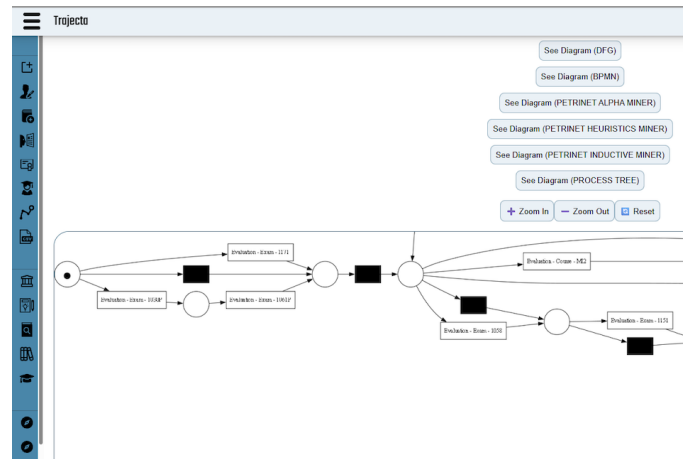


Fig. 7. Ejemplo funcionalidad Ver Diagramas de Log, con un diagrama de PetriNet

los comportamientos observados en el log de eventos, a forma de ejemplo, un modelo puede tener un fitness perfecto (todas las trazas del log son ejecutables en el modelo), pero aun así, permitir trazas que no se encuentran en el log de eventos, o que no serian posibles con las reglas reales del sistema. Midiéndose como precisión alta cuando el modelo SOLO permite lo que se observa en el log de eventos, y existen pocas desviaciones posibles que no se encuentran en el log de eventos mismo.

B. Implementación

Los 8 microservicios creados son:

- 1) Discoverer: Este microservicio se encarga de subir un log y de crear un thread que inicia el descubrimiento al mismo con diferentes algoritmos de Minería de Pro-

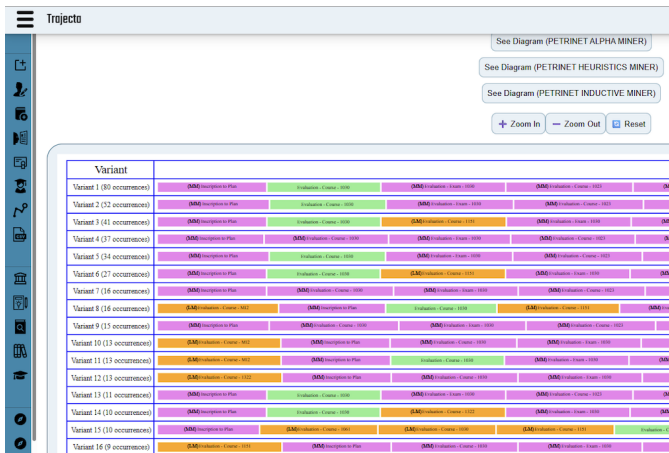


Fig. 8. Ejemplo funcionalidad ver Diagrama de alineamiento



Fig. 9. Ejemplo funcionalidad estadísticas de unidad curricular

cesos, tanto para los modelos de estudiantes como para los de referencia.

- 2) List Discovers - Se encarga de brindar y listar los ID de todos los descubrimientos realizados, permitiendo filtrar entre si es un descubrimiento del modelo de estudiantes, o si es un modelo de referencia.
- 3) Viewer - Se encarga de generar diagramas de los descubrimientos realizados, en diferentes formatos permitidos como petri-net, grafo de seguimiento directo, o en BPMN entre otros, como también poder ver los diagramas de rendimiento.
- 4) Aligement Viewer - Se encarga de realizar el análisis de alineamiento descrito anteriormente. Permitiendo ver cuales son las desviaciones que realicen los estudiantes. Presentándolo como un diagrama en forma de tabla.
- 5) Plan Stats - Brinda las estadísticas de un plan de un descubrimiento realizado. Con datos por ejemplo, de la

distribución de créditos de los estudiantes del plan.

- 6) Curricular Unit Stats - Brinda estadísticas de una unidad curricular de un descubrimiento con datos como los que fueron mencionados anteriormente.
- 7) Discover Stats - Brinda las estadísticas mencionadas anteriormente de los descubrimientos realizados.
- 8) Conformance Test - Realiza un analisis de conformidad usando Replay Token on a Petri Net entre un log y un modelo de referencia, y retorna indicadores y los datos necesarios para poder realizar un diagrama de conformidad. Junto con las metricas mencionadas.

Los microservicios se comunican a traves del FileSystem del sistema, siendo que, de cada archivo guardado, unicamente un único microservicio escribe en el, y los demás unicamente pueden leerlo, como es el caso del micro servicio Discoverer, el cual realiza un descubrimiento a un log y lo guarda en el disco para que luego otros micro servicios (por ejemplo el viewer), puedan leerlo y así trabajar con el.

Esto permite que por ejemplo, utilizando herramientas o soluciones que permitan tener sincronizado remotamente una carpeta o disco del FileSystem (por ejemplo usando un Raid en red), los micro servicios pueden estar ejecutando en diferentes servidores sin ninguna problemática.

VI. CONCLUSIONES

En conclusión, esta extensión desarrollada para la plataforma brinda un valor agregado en términos de usabilidad y análisis, al integrar técnicas de minería de procesos que permiten comprender de manera más clara las situaciones académicas de los planes de estudio, las trayectorias de los estudiantes y las problemáticas en unidades curriculares específicas. Los análisis generados permiten identificar puntos críticos, desvíos y dificultades en el recorrido académico, ofreciendo información útil para orientar mejoras.

Un aspecto importante de esta propuesta es que facilita el acceso al conocimiento generado mediante minería de procesos, ya que muestra resultados complejos de forma comprensible y sin necesidad de conocimientos técnicos avanzados. En particular, el uso del análisis de alineamiento permite identificar de manera explícita las desviaciones entre el modelo de referencia y el comportamiento real de los estudiantes, lo que hace posible detectar incoherencias y cuellos de botella, además de diseñar estrategias de mejora basadas en evidencia.

De esta manera, la extensión no se limita a mostrar información descriptiva, sino que funciona como un puente entre la teoría de la minería de procesos y su aplicación práctica en el ámbito educativo. La herramienta puede ser utilizada por docentes, autoridades y estudiantes, contribuyendo a un mayor entendimiento de la dinámica de los planes de estudio y apoyando decisiones orientadas a mejorar las currículas de las diferentes carreras y la experiencia académica de los estudiantes.

También, el análisis integral de las trayectorias estudiantiles mejora la planificación a largo plazo, posibilitando un seguimiento más personalizado y aportando flexibilidad en la gestión académica.

En definitiva, el proyecto logró ofrecer una solución completa que no solo atiende las limitaciones de otras herramientas, sino que además brinda beneficios concretos al mejorar la toma de decisiones y la planificación académica. La aplicación de la minería de procesos se muestra así como una estrategia efectiva para el seguimiento y gestión de datos educativos, con impacto directo en la calidad de los planes de estudio y en el acompañamiento de las trayectorias de los estudiantes.

REFERENCES

- [1] Informe proyecto inicial: https://github.com/fapret/Trajecta/blob/main/proyecto/informe_gr04.pdf
- [2] A Systematic Review on Process Mining for Curricular Analysis - Daniel Calegari — Andrea Delgado
- [3] pm4py: <https://processintelligence.solutions/pm4py>
- [4] OpenAPI Initiative - <https://www.openapis.org>
- [5] Eclipse Modeling Framework (EMF) - <https://eclipse.dev/modeling/emf>
- [6] XML Metadata Interchange - <https://www.omg.org/spec/XMI>
- [7] Java Servlets - <https://jakarta.ee/specifications/servlet/>
- [8] vis.js - <https://visjs.org/>
- [9] choices.js - <https://github.com/Choices-js/Choices>
- [10] Flask - <https://flask.palletsprojects.com/en/stable/>